TECHNICAL SPECIFICATION — SIMANTIK

Sistem Manajemen Layanan Teknologi Informasi

Versi: 1.0
Status: Baseline for Implementation
Tanggal: 7 Oktober 2026
Produk: SIMANTIK — Sistem Manajemen Layanan Teknologi Informasi
Organisasi: Diskominfo Pemerintah Daerah Kabupaten Bandung

---

1. Purpose

Dokumen ini menerjemahkan `docs/PRD.md` menjadi aturan teknis implementasi SIMANTIK.

Dokumen ini menjadi acuan:

- arsitektur aplikasi;
- struktur project;
- database;
- API;
- authentication;
- authorization;
- storage;
- workflow;
- frontend conventions;
- backend conventions;
- testing;
- security;
- implementation boundaries.

`PRD.md` menjelaskan **apa yang harus dilakukan produk**.

`TECHNICAL-SPEC.md` menjelaskan **bagaimana produk tersebut diimplementasikan**.

Apabila terdapat konflik dengan requirement organisasi/SOP resmi yang kemudian dikonfirmasi, requirement organisasi yang telah disetujui menjadi prioritas.

---

2. Source of Truth

Urutan prioritas:

1. Requirement/SOP/kebijakan resmi Diskominfo yang telah dikonfirmasi.
2. Peraturan perundang-undangan yang berlaku.
3. `docs/PRD.md`.
4. `docs/TECHNICAL-SPEC.md`.
5. Detail implementasi.

AI/developer **tidak boleh mengarang kebijakan organisasi**.

Jika sebuah aturan belum diketahui:

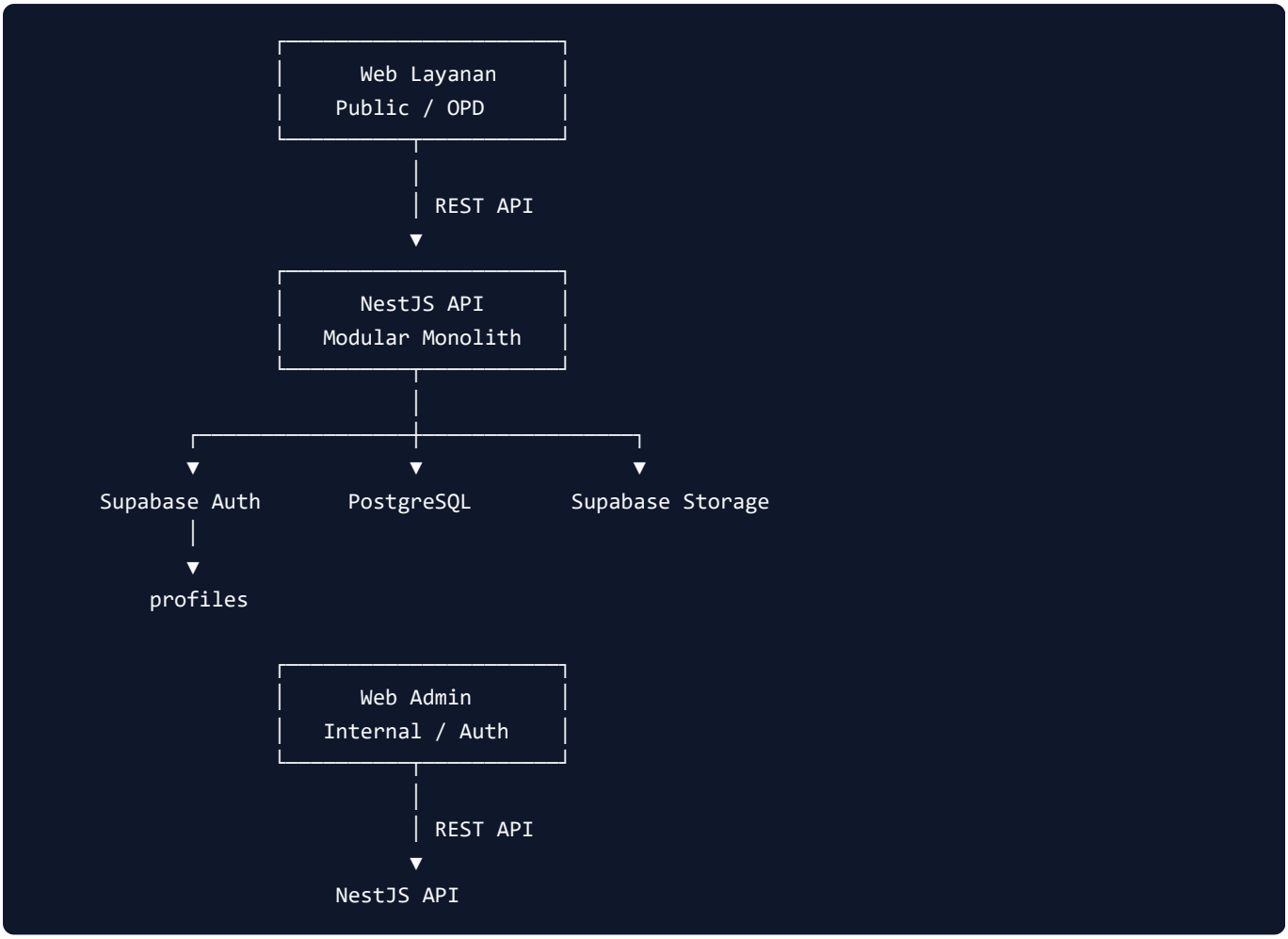
- tandai sebagai `OPEN QUESTION`; atau
- tandai sebagai `IMPLEMENTATION ASSUMPTION`.

Jangan mengubah assumption menjadi business rule resmi.

---

3. Architecture Summary

SIMANTIK menggunakan dua frontend yang terpisah dan satu backend API.



Architecture Decisions

- Frontend dan backend terpisah.
- Backend menggunakan NestJS.
- Backend berbentuk modular monolith.
- Database menggunakan Supabase PostgreSQL.
- Authentication menggunakan Supabase Auth.
- Storage menggunakan Supabase Storage private bucket.
- API menggunakan REST.
- API prefix: `/api/v1`.
- Tidak menggunakan ORM pada MVP.
- Database schema menggunakan SQL migrations.
- Data access backend menggunakan Supabase JS.
- TanStack Query digunakan untuk server state pada frontend.
- React Hook Form + Zod digunakan untuk form validation pada frontend.
- Motion digunakan untuk interaction/animation.
- Chat MVP menggunakan polling, bukan requirement WebSocket.

- Resend digunakan untuk email notification.

---

4. Technology Stack

4.1 Web Layanan

- React
- TypeScript
- Vite
- Tailwind CSS
- React Router
- TanStack Query
- React Hook Form
- Zod
- Motion

4.2 Web Admin

- React
- TypeScript
- Vite
- Tailwind CSS
- React Router
- TanStack Query
- React Hook Form
- Zod
- Motion
- `@supabase/supabase-js`

4.3 Backend

- Node.js
- TypeScript
- NestJS 12
- `@nestjs/config`
- `@supabase/supabase-js`

4.4 Infrastructure / Services

- Supabase PostgreSQL
- Supabase Auth
- Supabase Storage
- Resend

---

5. Repository Structure

Target structure:

```
simantik/
├── frontend/
│   ├── service-web/
│   │   ├── src/
│   │   ├── .env.local
│   │   └── .env.example
│   │
│   └── admin-web/
│       ├── src/
│       ├── .env.local
│       └── .env.example
│
├── backend/
│   └── api/
│       ├── src/
│       ├── .env
│       └── .env.example
│
├── supabase/
│   ├── migrations/
│   ├── seed/
│   └── tests/
│
├── docs/
│   ├── PRD.md
│   └── TECHNICAL-SPEC.md
│
├── .gitignore
├── package.json
└── package-lock.json
```

---

6. Frontend Architecture

6.1 General

Frontend harus menggunakan modular feature-oriented organization.

Hindari satu file yang berisi:

- seluruh page;
- seluruh form;
- seluruh API request;
- seluruh business logic;
- seluruh component.

Pisahkan:

```
pages
components
features
services
lib
hooks
types
```

sesuai kebutuhan aplikasi.

---

7. Web Layanan Frontend

Web Layanan tidak mempunyai login akun.

Contoh feature domains:

```
src/
├── app/
├── pages/
├── components/
├── features/
│   ├── services/
│   ├── application/
│   ├── tracking/
│   ├── messages/
│   └── help/
├── services/
├── hooks/
├── lib/
└── types/
```

Public API access harus selalu melewati NestJS API.

Browser tidak boleh menggunakan Supabase Secret Key.

---

8. Web Admin Frontend

Contoh domain structure:

```
src/
├── app/
├── pages/
├── components/
├── features/
│   ├── auth/
│   ├── dashboard/
│   ├── applications/
│   ├── assignments/
│   ├── workflow/
│   ├── lifecycle/
│   ├── testing/
│   ├── maintenance/
│   ├── evaluations/
│   ├── messages/
│   ├── notifications/
│   ├── audit/
│   ├── services/
│   └── technicians/
├── services/
├── hooks/
├── lib/
├── types/
└── router/
```

Struktur aktual dapat disesuaikan selama tetap modular.

---

9. Backend Architecture

NestJS menggunakan modular monolith.

Pattern:

```
Controller
    ↓
Service
    ↓
Repository / Data Access
    ↓
Supabase
```

Responsibilities

Controller

Menangani:

- HTTP request;
- route;
- DTO/input boundary;
- HTTP response.

Controller tidak boleh menampung business logic kompleks.

Service

Menangani:

- business rules;
- orchestration;
- authorization checks yang terkait business;
- workflow transitions;
- transaction boundaries.

Repository / Data Access

Menangani:

- query;
- insert;
- update;
- delete;
- select projection.

Guard

Menangani authentication dan authorization boundary.

---

10. Backend Module Boundaries

Target module:

```
src/
├── auth/
├── admin/
├── public/
├── applications/
├── workflow/
├── assignments/
├── lifecycle/
├── services/
├── messages/
├── notifications/
├── audit/
├── storage/
├── supabase/
├── config/
└── health/
```

Tidak semua folder harus dibuat sekaligus. Buat ketika domain tersebut mulai diimplementasikan.

---

11. Existing Backend Baseline

Backend saat ini telah berhasil:

- menjalankan NestJS;
- memiliki `/api/v1/health`;
- terhubung dengan Supabase service;
- memiliki `AuthModule`;
- memiliki `AuthGuard`;
- memiliki `/api/v1/admin/me`;
- berhasil mengautentikasi akun Admin;
- berhasil membaca `profiles.role`;
- berhasil login ke Dashboard Admin.

Baseline ini **tidak boleh dirusak** oleh implementasi feature berikutnya.

---

12. Environment Variables

Web Layanan

```
VITE_API_BASE_URL=http://localhost:3000/api/v1
```

Tidak boleh menyimpan Secret Key.

Web Admin

```
VITE_API_BASE_URL=http://localhost:3000/api/v1
VITE_SUPABASE_URL=https://sxzfwjapdlgvxkcnufmw.supabase.co
VITE_SUPABASE_PUBLISHABLE_KEY=<publishable-key>
```

Backend

```
PORT=3000
CORS_ORIGINS=http://localhost:5173,http://localhost:5174

SUPABASE_URL=https://sxzfwjapdlgvxkcnufmw.supabase.co
SUPABASE_SECRET_KEY=<secret-key>
```

Security Rule

- Secret key hanya di backend.
- Jangan gunakan prefix `VITE_` untuk secret.
- `.env` dan `.env.local` tidak di-commit.
- `.env.example` berisi placeholder.

---

13. Authentication

Admin Authentication

Supabase Auth digunakan untuk email/password.

Flow:

```
Login Form
    ↓
supabase.auth.signInWithPassword()
    ↓
Supabase Session
    ↓
Access Token
    ↓
NestJS API
    ↓
AuthGuard
    ↓
supabase.auth.getUser(accessToken)
    ↓
profiles
    ↓
active profile + role
```

Public Applicant

Tidak memiliki akun.

Public access menggunakan:

- ticket number;
- tracking code;
- short-lived tracking session/token dari backend jika diperlukan.

---

14. Authorization / RBAC

Roles:

```
SUPER_ADMIN
TECHNICIAN
```

SUPER_ADMIN

Akses penuh ke fungsi administrasi.

TECHNICIAN

Akses dibatasi:

- assignment;
- application scope;

- lifecycle scope;
- testing;
- maintenance;
- messages;
- fitur lain sesuai kewenangan.

Enforcement

Authorization harus dilakukan server-side.

Jangan mengandalkan:

```
React route guard saja
```

Frontend hanya digunakan untuk UX.

---

15. Authentication vs Authorization

Authentication menjawab:

> Siapa pengguna ini?

Authorization menjawab:

> Apa yang boleh dilakukan pengguna ini?

Keduanya harus dipisahkan.

Contoh:

```
Valid JWT
+
Profile aktif
+
Role TECHNICIAN
+
Application assigned kepada technician
```

baru dapat menghasilkan akses yang sah terhadap resource tertentu.

---

16. Database Baseline

Migration yang sudah diterapkan:

```
20261007102923_initial_schema.sql
20261007103425_applications.sql
20261007103947_workflow.sql
20261007104344_lifecycle.sql
20261007104547_communication_audit.sql
20261007104906_security_rls.sql
20261007105722_storage.sql
```

Jangan membuat ulang tabel yang sudah ada.

17. Current Database Domains

Master Data

```
opd_units
profiles
services
service_requirements
```

Application

```
applicants
applications
application_documents
```

Workflow

```
application_status_history
application_stage_history
assignments
```

Lifecycle

```
project_plans
technical_documents
test_records
maintenance_records
evaluations
```

Communication

```
messages
notifications
```

Audit

```
audit_logs
```

---

18. Database Rules

- Primary key menggunakan UUID.
- Timestamp menggunakan `timestamptz`.
- Foreign key digunakan untuk relational integrity.
- Master data dapat memiliki `is_active`.
- `updated_at` menggunakan trigger `set_updated_at`.
- Query list harus menggunakan pagination.
- Gunakan index untuk foreign key dan kolom pencarian/filter penting.
- Jangan menggunakan hard delete untuk histori penting.
- Audit/history tidak boleh dihapus sebagai efek normal dari UI.
- Migration adalah sumber perubahan schema.

---

19. Applications

`applications` adalah entity utama permohonan.

Konsep:

```
Application
├── ticket_number
├── tracking_code_hash
├── service
├── applicant
├── title
├── request details
├── current_status
├── current_stage
└── submitted_at
```

Tidak membuat tabel `tickets` terpisah pada MVP.

Ticket merupakan identitas operasional dari `applications`.

---

20. Applicant

`applicants` menyimpan identitas pemohon dan relasi OPD.

`applications` juga menyimpan snapshot identitas:

- applicant name;
- employee ID;
- position;
- email;
- phone.

Tujuan snapshot:

data histori pengajuan tetap merepresentasikan keadaan saat pengajuan dibuat.

---

21. Services

`services` berfungsi sebagai service catalog.

Minimal:

```
DEVELOPMENT
ENHANCEMENT
```

Namun label UI dapat menggunakan bahasa Indonesia yang lebih mudah dipahami.

`service_requirements` menentukan requirement masing-masing layanan.

---

22. Document Management

Applicant Documents

Metadata disimpan di:

```
application_documents
```

File fisik disimpan di bucket:

```
simantik-documents
```

Bucket:

- private;
- file size limit 10 MB;
- MIME allowlist sesuai migration.

Technical Documents

Metadata disimpan di:

```
technical_documents
```

Access

Jangan pernah memberikan public URL untuk dokumen privat.

Backend menentukan apakah requester berhak:

- melihat;
- mengunduh;
- mengunggah;
- mengganti versi.

23. Storage Path Convention

Target path:

```
applications/{application-id}/requirements/{file-id}
applications/{application-id}/technical/{file-id}
applications/{application-id}/testing/{file-id}
applications/{application-id}/maintenance/{file-id}
```

Nama file fisik harus menggunakan identifier aman, bukan nama upload mentah sebagai path utama.

Original filename tetap disimpan dalam metadata.

---

24. Application Submission Contract

Submission mengikuti:

```
Service
→ Applicant
→ Application Details
→ Documents
→ Terms
→ Review
→ Final Confirmation
→ Submit
```

Final submission harus atomic dari sudut pandang user.

Minimal prinsip:

- data validated;
- required documents validated;
- terms accepted;
- application created;
- ticket number created;
- tracking secret generated.

Jika salah satu bagian kritis gagal, sistem harus mencegah state setengah jadi.

---

25. Ticket Number

Ticket number:

- unique;
- human readable;
- tidak boleh dapat ditebak menjadi credential;
- tidak menjadi secret.

Format final ticket number masih `OPEN QUESTION` dan harus divalidasi organisasi.

Contoh format teknis sementara:

```
SIM-2026-000001
```

---

26. Tracking Code

Tracking code:

- diberikan kepada pemohon setelah successful submission;
- tidak disimpan plaintext di database;
- database menyimpan `tracking_code_hash`;
- harus cukup sulit ditebak;
- digunakan bersama ticket number.

Tracking code berfungsi sebagai secret untuk akses public tracking.

---

27. Public Tracking Security

Backend melakukan:

```
ticket_number
+
tracking_code
      ↓
verify
      ↓
authorized tracking context
      ↓
public-safe application data
```

Public API tidak boleh:

- menerima arbitrary application ID lalu mengembalikan detail;
- mengembalikan internal messages;
- mengembalikan audit log;
- mengembalikan private documents.

Gunakan DTO/view model khusus public.

Jangan menggunakan entity/database response mentah sebagai response public.

---

28. Tracking Session

Implementasi yang direkomendasikan:

1. pemohon mengirim ticket number + tracking code;
2. backend memverifikasi hash;
3. backend membuat short-lived tracking session/token;
4. request tracking berikutnya menggunakan session tersebut;
5. session memiliki expiration;
6. session tidak memberikan akses admin.

Detail session mechanism ditentukan dalam implementasi Auth Public.

---

29. Operational Status

Current enum:

```
SUBMITTED
VERIFICATION
REVISION_REQUIRED
APPROVED
REJECTED
IN_PROGRESS
COMPLETED
CLOSED
CANCELLED
```

Ini adalah technical design awal.

Status resmi organisasi belum dikonfirmasi.

---

30. Lifecycle Stage

Current enum:

```
ANALYSIS
PLANNING
DESIGN
IMPLEMENTATION
TESTING
MAINTENANCE
EVALUATION
```

Stage adalah posisi pekerjaan dalam lifecycle.

Stage bukan pengganti status.

---

31. Status Transition Authority

Semua status transition:

- hanya boleh dilakukan melalui backend;
- divalidasi berdasarkan current status;
- memeriksa role;
- memeriksa business rule;
- membuat `application_status_history`;
- mengisi actor;
- menyimpan note bila diperlukan.

Frontend tidak boleh bebas mengirim `current_status` sebagai update biasa.

UI harus memanggil endpoint action/transition yang sesuai.

---

32. Stage Transition Authority

Semua stage transition:

- melalui backend;
- divalidasi;
- membuat `application_stage_history`;
- menyimpan actor;
- menyimpan note jika relevan.

Tidak boleh mengubah `current_stage` secara langsung dari generic PATCH endpoint.

---

33. Assignment

`assignments` memiliki:

- application;
- technician;
- assigned by;

- assigned at;
- unassigned at;
- current flag.

Constraint:

```
maksimal satu current assignment
per application
```

Assignment history tetap dipertahankan.

---

34. Lifecycle Modules

Analysis

Memuat kebutuhan/analisis.

Planning

Memuat perencanaan pekerjaan.

Design

Memuat rancang bangun.

Implementation

Memuat artefak implementasi dan dokumentasi.

Testing

Mencatat hasil test.

Maintenance

Mencatat maintenance.

Evaluation

Mencatat evaluasi.

Tidak memaksa semua application melewati setiap stage dengan urutan identik.

---

35. Testing

Test types:

```
FUNCTION
INTEGRATION
LOAD
SECURITY
```

Result:

```
PASS
FAIL
PASS_WITH_NOTES
```

Minimum information:

- scope;
- criteria;
- scenario;
- expected result;
- actual result;
- finding;
- corrective action;
- tester;
- timestamp.

---

36. Maintenance

Types:

```
PERFECTIVE
ADAPTIVE
CORRECTIVE
PREVENTIVE
```

Minimum information:

- title;
- description;
- reason;
- action;

- result;
- start;
- completion;
- performer.

---

37. Evaluation

Evaluation minimum:

- evaluation date;
- success indicator;
- achievement;
- findings;
- recommendations;
- follow-up;
- evaluator.

---

38. Messages

`messages` selalu scoped ke:

```
application_id
```

Message visibility:

```
PUBLIC
INTERNAL
```

Sender types:

```
PUBLIC_APPLICANT
ADMIN
TECHNICIAN
SYSTEM
```

---

39. Public Message Rules

Public applicant hanya boleh:

- melihat public messages;
- mengirim public messages;
- mengakses messages dalam authorized tracking context.

Applicant tidak boleh:

- melihat internal messages;
- mengubah messages milik internal;
- menyatakan dirinya sebagai internal sender.

---

40. Internal Message Rules

Internal message:

- hanya dapat diakses authenticated internal user;
- hanya dapat dilihat sesuai application authorization;
- tidak boleh masuk public response DTO.

---

41. Chat Strategy

MVP:

```
GET messages
+
POST message
+
periodic polling
```

WebSocket tidak wajib.

Polling interval harus dibuat configurable dan tidak terlalu agresif.

Contoh:

```
5–10 seconds
```

dapat menjadi implementation starting point.

---

42. Notifications

Notification types:

```
APPLICATION_SUBMITTED
APPLICATION_STATUS_CHANGED
APPLICATION_STAGE_CHANGED
REVISION_REQUIRED
APPLICATION_ASSIGNED
NEW_MESSAGE
TEST_RESULT_UPDATED
APPLICATION_COMPLETED
APPLICATION_CLOSED
SYSTEM
```

Channels:

```
IN_APP
EMAIL
```

Delivery:

```
PENDING
SENT
FAILED
```

---

43. Email

Resend hanya digunakan dari backend.

API key Resend:

- server-side only;
- tidak dimasukkan ke Vite environment;
- tidak dikirim ke browser.

Email sending harus berada di notification service.

Business modules tidak seharusnya memanggil Resend secara langsung.

---

44. Audit

Audit event minimal:

- status transition;
- stage transition;
- assignment;
- important application data change;
- document action;
- testing update;
- maintenance update;
- evaluation update;
- administrative actions;
- closing.

Audit harus dibuat server-side.

Jangan menerima actor identity dari frontend sebagai sumber kebenaran.

Actor harus berasal dari authenticated context.

---

45. RLS Strategy

RLS telah di-enable pada seluruh tabel SIMANTIK.

Client roles:

```
anon
authenticated
```

tidak diberikan direct table access oleh migration security baseline.

Artinya:

```
Browser
   ×
Direct DB table access

Browser
   ↓
NestJS API
   ↓
Supabase
```

Backend menggunakan server credential.

RLS tetap dipertahankan sebagai defense-in-depth.

---

46. Storage Security

Bucket:

```
simantik-documents
```

bersifat private.

Browser tidak boleh menerima direct unrestricted access.

Backend melakukan authorization sebelum memberikan access.

Untuk kebutuhan download, gunakan mekanisme secure access/signed URL jika sesuai dengan arsitektur implementation.

47. API Conventions

Base URL:

```
/api/v1
```

Standard success:

```
{
  "success": true,
  "data": {},
  "meta": {}
}
```

Standard error:

```
{
  "success": false,
  "error": {
    "code": "ERROR_CODE",
    "message": "Human readable message",
    "details": {}
  }
}
```

48. HTTP Status Conventions

Gunakan secara konsisten:

```
200 OK
201 Created
204 No Content
400 Bad Request
401 Unauthorized
403 Forbidden
404 Not Found
409 Conflict
422 Unprocessable Entity
429 Too Many Requests
500 Internal Server Error
```

Tidak semua endpoint wajib menggunakan seluruh status tersebut.

---

49. Error Codes

Contoh:

```
AUTH_REQUIRED
AUTH_INVALID_TOKEN
AUTH_PROFILE_INACTIVE
AUTH_ROLE_FORBIDDEN

APPLICATION_NOT_FOUND
APPLICATION_INVALID_STATUS_TRANSITION
APPLICATION_INVALID_STAGE_TRANSITION

TRACKING_INVALID_CREDENTIAL
TRACKING_SESSION_EXPIRED

DOCUMENT_REQUIRED
DOCUMENT_INVALID_TYPE
DOCUMENT_TOO_LARGE

ASSIGNMENT_NOT_ALLOWED
MESSAGE_INTERNAL_FORBIDDEN
```

Error code harus stabil agar frontend dapat melakukan handling berdasarkan code, bukan parsing human message.

---

50. Public API

Planned:

```
GET    /api/v1/public/services
GET    /api/v1/public/services/:slug

POST   /api/v1/public/applications

POST   /api/v1/public/tracking/verify
GET    /api/v1/public/tickets/:ticket_number
GET    /api/v1/public/tickets/:ticket_number/timeline

GET    /api/v1/public/tickets/:ticket_number/messages
POST   /api/v1/public/tickets/:ticket_number/messages

POST   /api/v1/public/tickets/:ticket_number/revision
```

Public ticket endpoints wajib menggunakan authorized tracking context.

---

51. Admin API

Planned:

```
GET    /api/v1/admin/me

GET    /api/v1/admin/applications
GET    /api/v1/admin/applications/:id

POST   /api/v1/admin/applications/:id/verify
POST   /api/v1/admin/applications/:id/transitions
POST   /api/v1/admin/applications/:id/assignments

GET    /api/v1/admin/applications/:id/messages
POST   /api/v1/admin/applications/:id/messages
```

Tambahkan lifecycle-specific endpoints sesuai implementasi:

```
/project-plans
/technical-documents
/tests
/maintenance
/evaluations
```

Endpoint final harus konsisten terhadap domain ownership.

---

52. Technician API

Planned:

```
GET    /api/v1/technician/tasks
GET    /api/v1/technician/applications/:id

PATCH  /api/v1/technician/applications/:id

POST   /api/v1/technician/applications/:id/transitions

POST   /api/v1/technician/applications/:id/tests
POST   /api/v1/technician/applications/:id/maintenance
POST   /api/v1/technician/applications/:id/evaluations

GET    /api/v1/technician/applications/:id/messages
POST   /api/v1/technician/applications/:id/messages
```

Technician endpoint harus memeriksa assignment/permission server-side.

---

53. Service / Master Data API

SUPER_ADMIN:

```
GET    /api/v1/admin/services
POST   /api/v1/admin/services
PATCH  /api/v1/admin/services/:id

GET    /api/v1/admin/services/:id/requirements
POST   /api/v1/admin/services/:id/requirements
PATCH  /api/v1/admin/services/:id/requirements/:requirementId

GET    /api/v1/admin/opd-units
POST   /api/v1/admin/opd-units
PATCH  /api/v1/admin/opd-units/:id
```

Soft deactivation lebih diutamakan daripada hard delete untuk master data yang telah dipakai.

---

54. Technician Account Management

SUPER_ADMIN dapat:

- melihat technician;
- membuat account;
- mengaktifkan/nonaktifkan account;
- mengubah profile dasar;
- melihat status account.

MVP tidak perlu membuat custom password storage.

Credential management tetap menggunakan Supabase Auth.

---

55. Frontend API Client

Semua request ke backend menggunakan satu API client abstraction.

API client bertanggung jawab untuk:

- base URL;
- headers;
- JSON parsing;
- common error handling;
- authentication token injection untuk Admin;
- timeout strategy bila diperlukan.

Hindari `fetch()` tersebar di semua component.

---

56. Admin Token Handling

Admin browser:

```
Supabase Auth session
      ↓
access_token
      ↓
API client
      ↓
Authorization: Bearer <token>
```

API client harus mengambil session yang valid sebelum request protected.

Jangan menyimpan secret key.

---

57. Query Management

TanStack Query digunakan untuk:

- application list;
- application detail;

- services;
- notifications;
- messages;
- lifecycle data;
- dashboard stats.

Gunakan query keys yang konsisten.

Mutations digunakan untuk:

- transition;
- assignment;
- upload;
- message;
- testing;
- maintenance;
- evaluation.

---

58. Form Management

React Hook Form + Zod.

Semua form:

- memiliki schema;
- memiliki explicit field errors;
- memiliki loading state;
- memiliki submit state;
- memiliki server error state;
- tidak menghilangkan data user setelah validation error.

---

59. UX States

Setiap async feature harus memperhatikan:

```
Initial
Loading
Success
Empty
Error
Retry
Disabled
Submitting
```

Hindari halaman yang terlihat "diam" ketika request berjalan.

---

60. Routing

Web Layanan

Contoh route:

```
/
 /layanan
 /layanan/:slug
 /ajukan
 /ajukan/:serviceSlug
 /tracking
 /tracking/:ticketNumber
 /bantuan
```

Web Admin

Contoh:

```
/login
/
 /applications
 /applications/:id
 /assignments
 /testing
 /maintenance
 /evaluations
 /messages
 /notifications
 /services
 /technicians
 /audit
 /profile
```

Route final dapat berubah mengikuti IA/UI.

Protected Admin routes membutuhkan authenticated session.

---

61. Admin Shell

Web Admin menggunakan:

```
Sidebar
Header
Main content
```

Sidebar:

- collapsible;
- responsive;
- menu berdasarkan role;
- active route state.

SUPER_ADMIN dan TECHNICIAN tidak harus melihat menu yang sama.

---

62. Dashboard Information Architecture

Dashboard dapat memuat:

```
Summary cards
Recent applications
Status distribution
Workload
Testing queue
Revision queue
Notification summary
```

Jangan menampilkan metrics yang belum memiliki sumber data yang reliable.

---

63. List Page Standard

Application list harus memiliki:

- search;
- filters;
- sort;

- pagination;
- clear status;
- row action;
- loading state;
- empty state;
- error state.

Filter harus diteruskan ke backend untuk dataset besar.

---

64. Application Detail IA

Application detail dapat dibagi:

```
Overview
Applicant
Service
Documents
Workflow
Timeline
Assignment
Lifecycle
Testing
Maintenance
Evaluation
Messages
Audit
```

Tab/section hanya ditampilkan sesuai kebutuhan.

Public-safe view harus berbeda dari internal admin detail.

---

65. Timeline

Admin timeline dapat menampilkan:

- status history;
- stage history;
- assignment;
- selected system events.

Public timeline hanya public-safe events.

Jangan reuse raw internal audit timeline sebagai public timeline.

---

66. Accessibility

UI harus mempertimbangkan:

- keyboard navigation;
- visible focus;
- sufficient contrast;
- semantic HTML;
- accessible labels;
- error association;
- reduced motion preference;
- touch target yang nyaman.

Animation tidak boleh menjadi satu-satunya penyampai informasi.

---

67. Animation

Motion digunakan secara purposeful:

- transition;
- reveal;
- hover feedback;
- modal;
- loading;
- state changes.

Hindari:

- excessive parallax;
- continuous distracting motion;
- random animation;
- animation untuk elemen yang tidak memiliki fungsi UX.

---

68. Visual Identity

Visual baseline:

```
Primary:
White
Navy

Secondary:
Blue

Semantic:
Success
Warning
Error
Information
```

Logo hierarchy:

```
SIMANTIK
Diskominfo
Pemerintah Daerah
```

Tidak harus equal prominence.

UI harus terasa seperti digital government service yang profesional.

---

69. File Upload Security

Backend wajib memvalidasi:

- MIME type;
- extension;
- size;
- ownership/scope;
- destination path.

Jangan hanya mempercayai extension dari browser.

MVP bucket limit:

```
10 MB / file
```

Tipe file awal:

```
PDF
DOC
DOCX
XLS
XLSX
JPG/JPEG
PNG
```

Allowlist dapat diperluas setelah requirement organisasi dikonfirmasi.

---

70. Data Validation

Validation harus dilakukan minimal pada:

- frontend;
- backend.

Frontend validation meningkatkan UX.

Backend validation adalah authority.

Jangan menganggap client validation cukup untuk security.

---

71. Transactions / Consistency

Operation yang mengubah beberapa state penting harus dipikirkan sebagai satu business operation.

Contoh status transition:

```
validate
→ update current status
→ insert history
→ audit
→ notification event
```

Implementasi harus menghindari keadaan:

```
tetapi history gagal
```

Untuk operasi yang membutuhkan atomicity, gunakan transaction mechanism yang sesuai Supabase/PostgreSQL daripada melakukan beberapa perubahan tanpa kontrol.

---

72. Idempotency

Endpoint yang dapat dipanggil ulang karena network retry harus mempertimbangkan idempotency.

Terutama:

- final submission;
- revision submission;
- notification delivery;
- file upload metadata creation.

Jangan membuat dua application hanya karena user melakukan double-click.

---

73. Concurrency

Backend harus menangani kemungkinan:

```
dan
Admin B
```

mengubah application yang sama.

Status transition harus memvalidasi current state saat update.

Assignment juga harus menghormati current assignment constraint.

---

74. Logging

Application log backend harus membantu debugging tanpa membocorkan:

- password;
- access token;
- secret key;
- tracking code plaintext;
- sensitive document content.

Gunakan structured/contextual logging ketika implementasi semakin besar.

---

75. Observability MVP

MVP cukup memiliki:

- health endpoint;
- application logs;
- audit logs;
- error logging.

Tidak perlu menambah observability platform kompleks kecuali memang dibutuhkan.

---

76. Testing Strategy

Frontend

Test:

- form validation;
- loading states;
- route behavior;
- role-specific UI;
- public tracking UI;
- error handling.

Backend

Test:

- auth guard;
- authorization;
- service business rules;
- status transitions;
- stage transitions;
- public tracking verification;
- application submission;
- document validation;
- assignment rules.

Integration

Minimum flow:

```
Public submission
→ ticket issued
→ tracking
→ Admin login
→ verification
→ assignment
→ technician update
→ testing
→ completion
→ public timeline
```

---

77. Security Test Cases

Minimal:

```
No token → 401
Invalid token → 401
Inactive profile → 401
Valid TECHNICIAN accessing unauthorized app → 403
Public access without valid tracking credential → reject
Public access with valid credential → public-safe data only
Internal message from public endpoint → reject
Private document without authorization → reject
```

---

78. API Security

API harus menerapkan:

- input validation;
- authentication;
- authorization;
- rate limiting where appropriate;
- safe error messages;
- payload limits;
- secure file handling;
- audit logging for important actions.

Public endpoints terutama perlu perhatian terhadap abuse karena tidak memiliki account identity.

79. Public Endpoint Abuse Protection

Untuk tracking/public chat/submission:

- rate limit;
- validation;
- request size limit;
- anti-automation measures bila diperlukan;
- avoid revealing whether a ticket number exists without sufficient proof;
- consistent error responses where appropriate.

Mekanisme final dapat disesuaikan setelah testing.

80. Database Migration Rules

Semua perubahan schema melalui:

```
supabase/migrations/
```

Workflow:

```
migration new
→ edit
→ dry-run
→ db push
→ verify
```

Jangan membuat schema drift melalui perubahan manual yang tidak tercatat.

Jika perubahan manual terlanjur dilakukan, buat reconciliation migration.

81. Existing Migrations Must Be Preserved

Migration yang sudah berhasil diterapkan adalah history.

Jangan:

- rename sembarangan migration yang sudah applied;
- menghapus migration applied;
- mengedit migration applied untuk mengubah production history.

Perubahan baru dibuat lewat migration baru.

---

82. Seed Data Strategy

MVP seed dapat digunakan untuk:

- sample services;
- sample service requirements;
- sample OPD;
- development data.

Jangan memasukkan:

- production secret;
- real password;
- sensitive applicant data.

SUPER_ADMIN production account dibuat melalui Auth/admin process, bukan password hardcoded di Git.

---

83. Regulatory Mapping

Technical system mendukung konsep lifecycle:

```
ANALYSIS
PLANNING
DESIGN
IMPLEMENTATION
TESTING
MAINTENANCE
EVALUATION
```

Dokumentasi sistem mendukung artefak lifecycle tersebut.

Testing mendukung:

```
FUNCTION
INTEGRATION
LOAD
SECURITY
```

Maintenance mendukung:

```
PERFECTIVE
ADAPTIVE
CORRECTIVE
PREVENTIVE
```

Hal tersebut dirancang selaras dengan struktur lifecycle/dokumentasi yang menjadi acuan regulasi yang digunakan proyek.

Catatan: kesesuaian teknis aplikasi dengan keseluruhan ketentuan regulasi tetap membutuhkan validasi konteks organisasi, Arsitektur SPBE Pemerintah Daerah, dan prosedur internal.

---

84. Open Questions

Belum boleh di-hardcode sebagai kebijakan resmi:

1. Status resmi organisasi.
2. Approval authority.
3. Verification authority.
4. SLA.
5. Service-specific workflow.
6. Required documents tiap layanan.
7. Ticket number format.
8. Tracking code format.
9. Assignment structure.
10. Notification templates.
11. Official helpdesk channels.
12. Handover procedure.
13. Registration procedure.
14. Source-code repository requirement.
15. Maintenance/evaluation policy.

---

85. Implementation Assumptions

Untuk MVP, selama belum dikonfirmasi:

```
A1:
Modular monolith digunakan sebagai deployment architecture.

A2:
Admin authentication menggunakan Supabase Auth.

A3:
OPD tidak memiliki account.

A4:
Tracking menggunakan ticket number + tracking code.

A5:
Chat MVP menggunakan polling.

A6:
Dokumen disimpan pada private Supabase Storage.

A7:
Backend menjadi authority untuk authorization.

A8:
Status dan status transition adalah technical design awal.

A9:
Stage dan stage transition adalah technical design awal.

A10:
Tidak ada separate ticket table pada MVP.
```

Assumption harus ditinjau ketika SOP/requirement resmi tersedia.

---

86. Antigravity Implementation Rules

Antigravity harus memperlakukan:

```
docs/PRD.md
docs/TECHNICAL-SPEC.md
```

sebagai source of truth.

Sebelum mengubah code:

1. Baca dokumen.
2. Identifikasi module yang relevan.
3. Jangan mengubah architecture tanpa alasan.
4. Jangan menghapus migration existing.
5. Jangan mengarang business rule.

6. Jangan memindahkan secret ke frontend.
7. Jangan membuat direct database access dari browser.
8. Jangan membuat giant file.
9. Reuse existing shared components/services.
10. Pertahankan API contract.

---

87. Antigravity Change Discipline

Untuk setiap task AI:

```
Understand
→ Inspect existing code
→ Plan
→ Implement
→ Typecheck
→ Build
→ Test
→ Summarize changes
```

AI tidak boleh mengasumsikan bahwa file yang disebut secara konseptual pasti ada.

AI wajib membaca struktur project aktual sebelum membuat atau memindahkan file.

---

88. UI/UX Anti-Slop Requirements

Antigravity harus menghindari:

- generic SaaS dashboard look;
- excessive gradients;
- meaningless glassmorphism;
- excessive rounded cards;
- random decorative icons;
- inconsistent spacing;
- overuse of animations;
- copywriting generik;
- visual hierarchy yang lemah.

UI harus konsisten dengan:

- government service context;
- formal credibility;

- clear information hierarchy;
- accessibility;
- responsive behavior.

UI/UX Pro Max akan digunakan sebagai design-system/UX aid setelah technical specification ini direview.

---

89. Feature Delivery Strategy

Feature dikerjakan secara vertical slice.

Contoh:

```
Application Submission
├── UI
├── API
├── DB
├── Validation
├── Storage
├── Error handling
└── Test
```

Jangan membuat UI besar tanpa backend contract.

Jangan membuat backend besar tanpa user flow yang jelas.

---

90. Recommended Build Sequence

Phase 1 — Foundation

- project conventions;
- shared UI foundations;
- API client;
- auth session;
- protected routes.

Phase 2 — Admin Shell

- sidebar;
- header;
- dashboard;
- role-aware navigation.

Phase 3 — Service Catalog

- services;
- requirements;
- master data.

Phase 4 — Public Submission

- service selection;
- form;
- documents;
- review;
- submission;
- ticket issuance.

Phase 5 — Admin Applications

- list;
- detail;
- verification;
- workflow.

Phase 6 — Assignment

- technician;
- assignment;
- task list.

Phase 7 — Lifecycle

- analysis;
- planning;
- design;
- implementation;
- testing;
- maintenance;
- evaluation.

Phase 8 — Communication

- messages;
- notifications;
- email.

Phase 9 — Audit / Security Hardening

- audit;
- public tracking hardening;
- storage authorization;
- security testing.

Phase 10 — Final QA

- integration testing;
- responsive testing;
- regression;
- performance checks;
- documentation;
- deployment.

---

91. Definition of Done

A feature is done only when:

- requirement is mapped to PRD;
- technical behavior is mapped to this spec;
- UI implemented;
- backend implemented;
- authorization verified;
- validation implemented;
- loading/error/empty states handled;
- responsive behavior checked;
- tests added where appropriate;
- build passes;
- no secret exposed;
- no unrelated architecture damage introduced.

92. Current Checkpoint

Current verified state:

```
PRD v1.0                    ✅
Supabase project linked     ✅
Initial schema              ✅
Applications schema         ✅
Workflow schema             ✅
Lifecycle schema            ✅
Communication/audit schema  ✅
RLS baseline                ✅
Private Storage bucket      ✅
Admin Auth                  ✅
NestJS AuthGuard            ✅
Admin /me endpoint          ✅
Admin login                 ✅
Admin Dashboard             ✅
```

Next implementation checkpoint:

```
TECHNICAL-SPEC review
        ↓
UI/UX Pro Max setup
        ↓
Antigravity setup
        ↓
Admin Shell production UI
```

93. Final Technical Principle

SIMANTIK harus tetap:

```
Simple enough to deliver in 1 month
+
Structured enough to maintain
+
Secure enough for internal use
+
Traceable enough for government workflow
+
Flexible enough for future SOP refinement
```

Status: TECHNICAL SPECIFICATION BASELINE